

面试手册

操作系统

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
操作系统	3
01. 进程和线程有什么区别？	3
02. 进程状态和上下文切换如何理解？	3
03. 常见 CPU 调度算法怎么比较？	4
04. fork、exec、wait 分别做什么？	5
05. 常见 IPC 方式如何选择？	5
06. 虚拟内存解决了什么问题？	6
07. 分页、分段和多级页表有什么区别？	7
08. TLB 是什么？TLB miss 等于缺页吗？	8
09. 缺页异常如何处理？什么是写时复制？	8
10. 页面置换算法和抖动是什么？	9
11. 互斥锁、自旋锁、读写锁怎么选？	10
12. 信号量和互斥锁、条件变量的关系？	10
13. 为什么条件变量要在 while 中等待？	11
14. 死锁的四个必要条件和处理方式？	12
15. 阻塞、非阻塞、同步和异步 I/O 怎么区分？	12
16. epoll 的 LT 与 ET 有什么区别？	13
17. 文件描述符、打开文件描述和 inode 的关系？	14
18. inode、目录和软硬链接怎么理解？	14
19. 文件系统日志为什么能帮助崩溃恢复？	15
20. 用户态、内核态、系统调用和中断的关系？	16

操作系统

从进程到持久化

以通用操作系统原理为主，结合 Linux/POSIX 接口讨论进程、内存、并发与 I/O；具体调度器和内核细节需注明版本。

01. 进程和线程有什么区别？

进程与调度 / 基础

进程组织地址空间和资源，线程组织执行流。同一进程的线程通常共享代码、堆和文件描述符，但各自保留寄存器、栈及调度状态；共享使通信方便，也引入同步问题。

回答要点

1. 进程间默认有地址空间隔离，某个进程的普通内存写入不会直接覆盖另一个进程；线程共享地址空间，因此一个线程的非法写入可能破坏整个进程的运行。
2. 每个线程都要保存程序计数器、寄存器和自己的栈，局部变量并非自动可被其他线程安全访问；通过指针共享栈对象还必须保证对象生命周期覆盖访问过程。
3. 比较创建与切换成本时应说明系统实现：同进程线程切换常可保留地址空间，跨进程可能切换页表并影响 TLB；缓存冷却和调度策略也会改变实际成本。

常见追问

追问：多线程一定比多进程快吗？答：不一定。线程减少部分资源开销，但共享数据会带来锁竞争和缓存争用；多进程隔离更强。应以任务粒度、通信量、故障隔离需求和实测结果选择。

易错点

易错点：把线程等同于轻量进程后就认为它没有独立栈，或认为线程切换完全不进入内核。用户态协程和内核线程的调度机制需要分别讨论。

延伸阅读 / 原始参考资料

OSTEP • Concurrency and Threads

Linux 手册 • pthreads(7)

02. 进程状态和上下文切换如何理解？

进程与调度 / 基础

经典模型包括运行、就绪和阻塞。就绪表示具备运行条件但暂未获 CPU，阻塞表示等待事件。切换保存旧任务的执行现场并恢复新任务，事件完成通常先转为就绪。

回答要点

1. 时间片耗尽或更高优先级任务到达，可使运行任务回到就绪；等待 I/O、锁或条件的任务进入阻塞，不能只靠给它更多时间片让等待条件立即满足。
2. 上下文通常涉及寄存器、程序计数器、栈指针及内核维护的调度信息；跨地址空间切换还可能改变页表根，间接造成 TLB 和 CPU 缓存命中率变化。
3. 状态图是教学抽象，Linux 等系统还区分可中断睡眠、不可中断睡眠、停止与僵尸等状态；回答时先说明通用模型，再解释所讨论系统的扩展状态。

常见追问

追问：系统调用必然发生进程切换吗？答：不会。系统调用首先是同一任务从用户态进入内核态，处理完成后可能直接返回；只有阻塞、抢占或主动让出等条件触发调度时才切换任务。

易错点

易错点：混淆模式切换和上下文切换；I/O 完成也不意味着该任务马上占用 CPU，它通常需要进入就绪队列等待调度器选择。

延伸阅读 / 原始参考资料

OSTEP • The Process

OSTEP • Limited Direct Execution

03. 常见 CPU 调度算法怎么比较？

进程与调度 / 基础

先来先服务简单但可能出现护航效应；短作业优先在已知运行时间的模型下改善平均周转；时间片轮转改善响应。实际调度需权衡公平、吞吐、响应和饥饿。

回答要点

1. FCFS 可让短任务被长任务阻挡；SJF 需要预测任务长度，抢占式版本按剩余时间选择任务。说明理论最优时必须同时给出到达时间和长度已知等模型假设。
2. 轮转使用时间片让就绪任务交替执行，较小时时间片有利于响应但增加切换成本；较大时间片逐渐接近 FCFS。时间片并非越小越好，应联系切换开销和交互需求。
3. 多级反馈队列根据历史行为调整优先级，周期性提升可缓解饥饿；真实 Linux 调度器随版本演进，不能把教学算法直接宣称为所有版本的实际实现。

常见追问

追问：周转时间和响应时间有什么区别？答：周转从到达到完成，响应从到达到首次获得响应；一个算法可能让首次响应很快，却因反复切换拉长总完成时间。面试时应先明确优化目标。

易错点

易错点：把高优先级等同于短作业，或声称短作业优先在任何实际系统中都最优。未知任务长度、公平性与实时约束会改变算法选择。

延伸阅读 / 原始参考资料

OSTEP • Scheduling

OSTEP • Multi-Level Feedback Queue

04. fork、exec、wait 分别做什么？

进程与调度 / 基础

POSIX 的 `fork` 创建子进程，`exec` 系列替换当前进程的程序映像，`wait` 系列获取子进程退出状态并回收相应记录。`fork` 后父子返回值不同，执行先后由调度决定。

回答要点

1. `fork` 后父进程得到子进程 PID，子进程得到零，失败则返回错误；父子拥有独立的逻辑地址空间。Linux 常用写时复制推迟数据页复制，页表等仍有管理开销。
2. `exec` 成功后不返回原调用点，而是运行新程序；PID 通常保持不变。文件描述符是否保留受 `close-on-exec` 标志影响，不能说 `exec` 无条件清空全部进程资源。
3. 子进程退出后保留状态供父进程读取，未回收会形成僵尸；孤儿指父进程先退出的存活子进程。两者描述不同维度，回收机制需结合系统和子进程管理方式。

常见追问

追问：为什么 shell 常先 `fork` 再 `exec`？答：shell 需要保留自身的命令解释循环，同时让子进程执行目标程序；还可以在子进程执行前设置重定向等环境，父进程按前台或后台语义决定是否等待。

易错点

易错点：`exec` 创建新进程、僵尸还在占用 CPU、父进程退出后子进程必然立即退出，这三个说法都不准确。应区分程序替换、退出状态保存与进程收养。

延伸阅读 / 原始参考资料

OSTEP • Process API

Linux 手册 • `fork(2)`

Linux 手册 • `execve(2)`

05. 常见 IPC 方式如何选择？

进程与调度 / 基础

IPC 可通过管道、消息队列、共享内存、信号和套接字完成。选择时关注是否跨机器、消息边界、数据规模和同步需求；共享内存省去部分复制，却需要自行约束并发访问。

回答要点

1. 管道适合字节流，普通匿名管道常用于有亲缘关系的进程；FIFO 可通过路径让无亲缘进程通信。读写端关闭、阻塞规则和缓冲容量都应纳入协议设计。
2. 共享内存让多个地址空间映射同一存储区域，适合大数据量交换；数据布局、初始化、对象生命周期和同步必须有约定，映射共享区域并不会自动提供互斥。
3. 消息队列强调离散消息，套接字可实现本机或跨网络通信，信号更适合有限事件通知；不同机制支持的边界、持久性和权限模型不同，应按问题选择。

常见追问

追问：共享内存为什么还要加锁？答：多个进程可能同时修改同一队列索引或读取尚未发布的数据；共享只解决可访问性，锁、信号量或正确设计的原子协议才负责互斥、顺序和可见性。

易错点

易错点：认为共享内存绝对没有任何复制开销，或把所有管道写入都当作原子消息。管道是字节流，写入原子性还受到 PIPE_BUF 等条件约束。

延伸阅读 / 原始参考资料

Linux 手册 • pipe(7)

Linux 手册 • shm_overview(7)

Linux 手册 • unix(7)

06. 虚拟内存解决了什么问题？

内存管理 / 基础

虚拟内存为每个进程提供逻辑地址空间，通过地址转换和权限检查映射到物理资源。它支持隔离、共享、按需分配和换出；核心并非单纯把磁盘当作更大的内存。

回答要点

1. 程序使用虚拟地址，硬件与内核依据页表完成转换并检查读写执行权限；不同进程可以使用相同虚拟地址映射到不同物理页，简化程序装载并实现隔离。
2. 代码、堆、栈和映射文件组成地址空间，各区域的权限与生命周期不同；虚拟区域被预留或映射，并不保证每一页立即分配物理内存，更不保证未来分配必成功。
3. 共享库、共享内存和写时复制可让多个虚拟映射引用同一物理页；是否能写、何时复制和何时回收由映射属性决定，隔离与共享可以同时存在于一个系统。

常见追问

追问：64 位程序就能使用全部 2 的 64 次方字节吗？答：不意味着如此。CPU 实际实现的虚拟地址位宽、操作系统布局、进程限制和可用物理资源都会限制地址范围及可成功分配的容量。

易错点

易错点：把虚拟内存大小、驻留集大小和实际物理占用混为一谈。讨论内存不足时应同时检查映射、驻留页、共享页和系统提交策略。

延伸阅读 / 原始参考资料

OSTEP • Address Spaces

OSTEP • Address Translation

07. 分页、分段和多级页表有什么区别？

内存管理 / 基础

分页使用固定大小页和页框，分段依据代码、数据等逻辑区域划分可变大小段。多级页表把页表自身按层组织，稀疏地址空间可只建立需要的下级表，但增加查表层次。

回答要点

1. 分页将地址拆成虚拟页号与页内偏移，转换只替换页号，偏移保持不变；固定页框减少传统外部碎片，但末页未用空间仍形成内部碎片。
2. 分段为不同逻辑区域配置基址、界限和权限，表达逻辑结构直观；可变大小区域的分配容易出现外部碎片。现代体系也可能结合分段和分页，不能只用二选一模型。
3. 多级页表按地址位段逐层索引，未使用的大地址区域无需分配完整叶表；代价是未命中 TLB 时的页表遍历。大页可减少层级或条目，但会带来分配与碎片权衡。

常见追问

追问：4 KiB 页的偏移需要多少位？答：需要十二位，因为 4096 等于 2 的 12 次方；地址其余有效位用于页号和页表层级索引。具体各级索引位数由所讨论的体系结构和页表格式决定。

易错点

易错点：认为多级页表一定比单级页表占用更少内存。它主要适合稀疏地址空间；若地址空间密集使用，上层索引结构也需要额外空间。

延伸阅读 / 原始参考资料

OSTEP • Paging

OSTEP • Advanced Page Tables

OSTEP • Segmentation

08. TLB 是什么？TLB miss 等于缺页吗？

内存管理 / 基础

TLB 缓存最近使用的地址转换及相关权限，减少查页表成本。TLB miss 表示缓存没有该转换；页表仍可能指向驻留物理页。缺页是地址访问触发的异常，两者不能混同。

回答要点

1. TLB 命中时可快速得到物理页号；未命中时由硬件或软件遍历页表补充转换，机制依体系结构而异。页表遍历本身可能访问缓存和内存，因此会引入延迟。
2. 进程切换后旧转换不能误用于新地址空间，可通过刷新或地址空间标识区分；映射被修改时还必须使相关旧转换失效，多核系统需要协调相关 CPU 的 TLB。
3. TLB 容量有限，跨大量页面的随机访问可能使转换频繁失效；大页提高一次转换覆盖的范围，但增加内存分配难度，并可能浪费较小对象周围的空间。

常见追问

追问：数据全部在 CPU cache 中，为什么访问仍然慢？答：数据缓存命中不代表地址转换缓存命中。如果访问跨越很多页，TLB miss 及页表遍历仍可成为开销，应结合数据访问和地址转换的指标分析。

易错点

易错点：把 TLB 说成保存完整数据页的缓存，或者把任何 TLB miss 都解释成从磁盘读页。它主要缓存地址转换，实际缺页还要检查页表状态。

延伸阅读 / 原始参考资料

OSTEP • Translation Lookaside Buffers

09. 缺页异常如何处理？什么是写时复制？

内存管理 / 进阶

缺页处理先判断地址和权限是否有效，再决定分配零页、加载文件或换入页面；无效访问可能被终止。写时复制让页面先共享，发生写入后才为写入方建立私有副本。

回答要点

1. 合法地址未驻留时，内核可以分配物理页并填充数据，更新页表后让指令重试；读取磁盘只是其中一种路径，首次访问零填充页也可能发生缺页而无需磁盘读取。
2. 权限异常可能来自真正非法写入，也可能是写时复制的保护安排；内核需要查看映射属性，不能把所有保护异常都判定为程序错误，也不能无条件允许写入。

3. fork 后共享的可写数据页常暂时标为只读，并记录写时复制关系；任一进程写入时建立私有页，其他进程保持原内容。引用计数和并发处理决定页面何时可回收。

常见追问

追问：minor fault 和 major fault 怎么区分？答：Linux 统计中主要看处理该缺页是否需要相关存储 I/O；轻微缺页也可能涉及分配、页表更新和复制，因此数量高仍会耗费 CPU，不能认为完全免费。

易错点

易错点：malloc 成功不等于所有页面已经驻留；缺页不等于非法访问。应区分地址有效性、页面驻留状态、权限保护和具体后备存储。

延伸阅读 / 原始参考资料

OSTEP • Swapping Mechanisms

Linux 手册 • getrusage(2)

Linux 手册 • mmap(2)

10. 页面置换算法和抖动是什么？

内存管理 / 进阶

页面置换在物理内存紧张时选择牺牲页，目标是减少未来缺页。最优算法需要未来访问序列，LRU 用过去近似未来，Clock 用访问位降低维护成本；频繁换入换出称为抖动。

回答要点

1. FIFO 按进入时间淘汰，未必照顾热点，可能出现分配更多页框却缺页更多的 Belady 异常；真正的 LRU 属于栈算法，但精确记录每次访存顺序的成本较高。
2. Clock 环形扫描页面并参考访问位，给近期使用的页第二次机会；实际系统还区分匿名页、文件页、脏页以及回收成本，不能用单一教学算法概括全部实现。
3. 当活跃工作集超过可提供的内存时，刚换入的页可能很快再次被淘汰，I/O 时间占比升高；降低并发、优化局部性或增加资源可能比只调整置换算法有效。

常见追问

追问：缓存命中率高为什么仍会卡顿？答：缺页的代价差别很大，少量需要存储 I/O 的缺页可能主导尾延迟；还要观察换页频率、工作集、存储延迟和内存压力，不能只看平均命中率。

易错点

易错点：把操作系统实际回收策略直接说成精确 LRU，或认为加内存一定按线性比例提速。置换效果取决于访问模式、工作集与页面后备类型。

延伸阅读 / 原始参考资料

OSTEP • Swapping Policies

11. 互斥锁、自旋锁、读写锁怎么选？

并发与同步 / 基础

互斥锁保护临界区，竞争时通常可以阻塞等待；自旋锁在等待时持续检查；读写锁允许多个读者但限制写者。选择取决于持锁时间、竞争程度、运行环境和公平性要求。

回答要点

1. 自旋适合预期很短且持锁者能尽快继续运行的等待；长时间持锁、单核竞争或持锁者被抢占时，自旋浪费 CPU。内核自旋锁还受到是否允许睡眠等上下文限制。
2. 阻塞式互斥锁可节省等待 CPU，但睡眠与唤醒产生调度成本；实际锁实现常结合用户态快速路径、短暂自旋和内核等待，不能把每次加锁都说成一次系统调用。
3. 读写锁在大量并发读取且写入较少时可能有效，但维护读者状态也有开销；写者饥饿、锁升级和长读临界区都需分析，读多并不自动意味着读写锁更快。

常见追问

追问：临界区只有一个自增为什么还要同步？答：普通自增通常包含读、修改和写，多线程交错会丢失更新。可以使用锁或适当原子操作；正确的选择还要考虑被保护的不变量是否涉及多个变量。

易错点

易错点：自旋锁不是忙等越久性能越好；读写锁也不是万能的读多优化。首先缩短临界区、减少共享，再基于测量选择等待方式和锁粒度。

延伸阅读 / 原始参考资料

OSTEP • Locks

Linux 手册 • `pthread_mutex_lock(3)`

12. 信号量和互斥锁、条件变量的关系？

并发与同步 / 基础

信号量维护许可数量，等待操作获取许可，发布操作归还或增加许可。互斥锁表达临界区所有权，条件变量表达等待某个受锁保护的条件；三者语义不同，不能随意替换。

回答要点

1. 计数信号量可限制同时访问资源的任务数，初值对应许可数量；生产者消费者中可分别计数空位和已有元素，还需要互斥保护共享队列的索引与存储。
2. 二值信号量可以实现部分互斥效果，但通常没有互斥锁的同等所有权要求；互斥锁强调由持有者解锁，具体递归、错误检查和优先级继承能力依 API 而异。

3. 条件变量自身不保存资源计数，唤醒后还要重新检查谓词；信号量的许可可在没有等待者时累积。把条件通知当成可永久保存的事件，会导致丢失等待协议。

常见追问

追问：限制同时最多十个下载任务用什么？答：可用初值十的计数信号量，任务开始前获取许可，成功、失败或取消路径都归还许可；资源管理应保证异常情况下不泄漏许可，避免并发额度逐渐耗尽。

易错点

易错点：只设置一个表示队列非空的信号量，却没有处理容量和共享索引并发。计数、互斥和条件等待是不同职责，要按不变量组合。

延伸阅读 / 原始参考资料

OSTEP • Semaphores

Linux 手册 • sem_overview(7)

13. 为什么条件变量要在 while 中等待？

并发与同步 / 基础

条件变量等待必须与受互斥锁保护的谓词配合：持锁检查，条件不满足就等待，醒来后重新检查。while 处理虚假唤醒和其他线程抢先消耗条件，通知只表示可以重试。

回答要点

1. 等待操作需要原子地释放互斥锁并进入等待，再在返回前重新获得锁；这避免检查条件后、真正睡眠前发生通知而丢失唤醒。手动先解锁再睡眠无法保证同等协议。
2. 一次通知不意味着当前线程一定获得资源，其他消费者可能先拿走元素；广播会唤醒多个等待者，而可用资源可能有限。醒来必须重新确认谓词而不是直接继续。
3. 改变谓词的一方也应按约定持有同一锁，先更新共享状态再通知；多个不同等待条件可以使用多个条件变量，但仍要设计对应锁和谓词，避免无效唤醒和遗漏通知。

常见追问

追问：为什么用 if 会出错？答：消费者收到通知后还需竞争锁，在它获得锁前队列可能再次为空；if 只检查了睡眠前的状态。while 重检能让它安全继续等待，虚假唤醒同样适用这个理由。

易错点

易错点：认为通知次数就是资源数量，或者认为被通知线程直接获得执行权。通知、调度和获得互斥锁是不同步骤，程序正确性应建立在谓词上。

延伸阅读 / 原始参考资料

OSTEP • Condition Variables

Linux 手册 • pthread_cond_wait(3)

14. 死锁的四个必要条件和处理方式？

并发与同步 / 进阶

经典死锁需要互斥、占有并等待、不可抢占和循环等待同时成立。可以通过设计破坏条件、分配前避免不安全状态，或运行时检测后恢复；忙碌但无进展的活锁需另行区分。

回答要点

1. 统一锁顺序可破坏循环等待，例如所有路径先拿 A 再拿 B；一次申请全部资源可减少占有并等待，但限制并发。措施必须覆盖全部访问路径，包括错误恢复和回调。
2. 银行家算法根据最大需求判断状态是否安全，适用于需求可预先描述的资源模型；一般应用的锁依赖未必满足这个前提，不能把它作为无需约束的通用解决方案。
3. 检测等待图后可以终止任务、回滚事务或释放可恢复资源，但恢复动作有业务成本；超时能打断部分等待，也可能因频繁重试造成活锁，应设置退避并保证回滚完整。

常见追问

追问：一次观察到循环等待就一定是死锁吗？答：单实例资源且边代表确定等待时，等待图环是有力依据；多实例资源图的环不必然构成死锁。应说明图中节点、边和资源数量的模型。

易错点

易错点：把四个必要条件当成任意一个即可证明死锁，或认为加超时自动保证业务一致性。超时后的资源释放与已发生的操作仍需要正确处理。

延伸阅读 / 原始参考资料

OSTEP • Concurrency Bugs

15. 阻塞、非阻塞、同步和异步 I/O 怎么区分？

I/O 与文件系统 / 基础

阻塞描述调用是否等待才能返回，异步描述操作完成如何通知调用方，两者不是同一维度。非阻塞 read 无数据时可立即返回 EAGAIN；完成通知型异步 I/O 则在操作完成后报告结果。

回答要点

1. 阻塞 I/O 可让调用线程休眠直到数据或结果可用；非阻塞 I/O 要处理暂时不可操作、部分读写和重试。反复忙轮询虽然不阻塞调用，却可能消耗大量 CPU。
2. select、poll、epoll 主要通知文件描述符是否适合执行 I/O，通常随后还需 read 或 write；就绪通知与已完成读写不同，因此不能直接把 epoll 等同于完成通知异步 I/O。

3. 事件循环可以用少量线程管理大量等待中的连接，但 CPU 密集处理和阻塞文件操作仍可能阻塞整个循环；可把这类工作放到工作线程，并设计结果回传和背压。

常见追问

追问：epoll_wait 能阻塞，为什么还叫事件驱动？答：事件循环可以在没有就绪事件时休眠，有事件后统一处理；事件驱动指控制流按通知推进，并不要求每个调用都立即返回，也不代表完全无需线程。

易错点

易错点：把非阻塞等同于异步，把同步等同于始终阻塞。应先说明通知的是就绪还是完成，以及哪个阶段会等待、由谁执行数据传输。

延伸阅读 / 原始参考资料

OSTEP • Event-based Concurrency

Linux 手册 • aio(7)

Linux 手册 • read(2)

16. epoll 的 LT 与 ET 有什么区别？

I/O 与文件系统 / 进阶

Linux epoll 用兴趣集合和就绪集合管理事件。LT 在条件仍就绪时可再次通知，ET 主要在就绪变化时通知；ET 通常配合非阻塞描述符，处理到 EAGAIN 再等待。

回答要点

1. 如果一次事件到来后只读取部分数据，LT 可以继续报告可读，ET 却可能没有新变化可报告，导致剩余数据滞留；ET 处理器应循环读取，同时正确处理断开、错误和中断。
2. 非阻塞套接字的写入可能只接受部分数据，剩余内容要放进应用缓冲区；需要写事件时再关注可写就绪，避免常驻可写通知产生忙循环，并限制缓冲区实施背压。
3. epoll_wait 返回的事件数量影响单次处理工作量，注册、唤醒和应用处理也有开销；不能因为内部维护就绪集合，就声称所有操作、所有负载下都严格为 O(1)。

常见追问

追问：ET 会不会丢事件？答：常见问题是应用没有消耗到 EAGAIN 就再次等待，或者错误管理状态与重新武装；事件通知不能被当作字节计数。可使用明确的连接状态机，保证每次处理都能继续推进。

易错点

易错点：把 ET 描述为每个数据包恰好通知一次。通知可合并，且网络数据包与套接字可读字节流不是一一对应关系；应按就绪条件理解。

延伸阅读 / 原始参考资料

Linux 手册 • epoll(7)

Linux 手册 • [epoll_ctl\(2\)](#)

Linux 手册 • [epoll_wait\(2\)](#)

17. 文件描述符、打开文件描述和 inode 的关系？

I/O 与文件系统 / 基础

文件描述符是进程用于引用打开对象的整数；内核打开文件描述保存偏移与状态，inode 保存文件元数据。
dup 或 fork 可让多个描述符共享同一打开文件描述，因此也共享文件偏移。

回答要点

1. 描述符表属于进程资源视图，描述符数字可以在关闭后复用，不能把数字当作永久对象身份；读写套接字、管道和普通文件都可能通过文件描述符接口进行。
2. 同一文件被分别 open 两次，通常得到不同打开文件描述和独立偏移；dup 得到的新描述符引用同一描述，因此一个描述符读取后另一个会观察到推进的偏移。
3. inode 保存类型、权限、大小及数据定位等信息，目录负责把名字关联到 inode；文件名并不是 inode 必须保存的字段，多个硬链接可对应同一 inode。

常见追问

追问：fork 后父子都读同一个继承描述符会怎样？答：它们共享打开文件描述，因而共享偏移，读取会消耗同一个顺序位置；独立 open 或使用显式偏移的 pread 可以表达不同读取需求，并仍需考虑共享数据的变化。

易错点

易错点：把描述符表、系统打开文件描述和 inode 说成同一层，或认为 dup 会复制文件内容。它主要建立另一个引用，不创建独立文件副本。

延伸阅读 / 原始参考资料

OSTEP • [Files and Directories](#)

Linux 手册 • [open\(2\)](#)

Linux 手册 • [dup\(2\)](#)

18. inode、目录和软硬链接怎么理解？

I/O 与文件系统 / 基础

目录项把名字映射到文件标识，inode 描述文件自身。硬链接新增同一文件的名字，符号链接是保存路径的独立对象；删除一个名字不一定立刻释放文件内容。

回答要点

1. 硬链接指向同一 inode，多个名字读取的是同一份文件数据；常规文件系统通常不允许跨文件系统硬链接，并限制对目录创建硬链接，避免层次结构出现复杂循环。
2. 符号链接保存目标路径，目标可以不存在，也可跨文件系统；相对目标通常相对于符号链接所在目录解析。目标被移动或删除时可能形成悬空链接。
3. unlink 移除目录项并减少链接计数，文件还可能被打开文件描述引用；通常在链接计数归零且没有仍需保持对象的引用时，系统才最终回收相应数据与元数据。

常见追问

追问：删除正在写入的日志后磁盘空间为何没释放？答：进程可能仍持有已经没有目录名字的文件对象，继续通过描述符写入。关闭相关引用后空间才有机会回收，排查时要确认被删除文件的打开引用。

易错点

易错点：把符号链接说成另一个共享 inode 的文件名，或认为 unlink 必然立即销毁文件。目录项、文件对象和进程引用有各自的生命周期。

延伸阅读 / 原始参考资料

OSTEP • File System Implementation

Linux 手册 • link(2)

Linux 手册 • symlink(2)

Linux 手册 • unlink(2)

19. 文件系统日志为什么能帮助崩溃恢复？

I/O 与文件系统 / 进阶

文件系统更新可能涉及多个数据块与元数据块，崩溃会留下部分完成的状态。日志先记录事务意图或更新内容，再按规则提交和应用，使恢复能够重放或丢弃未完成事务。

回答要点

1. 例如扩展文件需要修改数据块、分配位图和 inode，任意写入顺序都可能在崩溃后产生不一致；日志把关联修改组织成事务，并通过提交标记判断是否应恢复。
2. 元数据日志与数据日志提供不同保证，不能认为所有日志文件系统都保护所有用户数据；写回缓存、设备持久化语义和写入排序也会影响断电后的结果。
3. write 成功通常说明数据已被系统接收，并不等于落到持久介质；需要持久化时按接口语义使用 fsync 等操作，涉及创建或重命名时还需考虑目录项的持久化。

常见追问

追问：用了日志还需要 fsync 吗？答：需要依据持久化要求判断。日志主要解决文件系统一致性，不自动保证每次应用写入都在断电前持久化；业务必须明确提交边界，并正确处理同步操作返回的错误。

易错点

易错点：把日志文件系统一致性、应用事务原子性和数据永不丢失混为一谈。它们是不同的层级的保证，需要说明日志模式和实际持久化接口。

延伸阅读 / 原始参考资料

OSTEP • Crash Consistency

Linux 手册 • fsync(2)

20. 用户态、内核态、系统调用和中断的关系？

I/O 与文件系统 / 基础

用户态限制特权操作，内核态允许管理受保护资源。系统调用通过受控入口请求内核服务，中断响应外部事件，异常来自执行过程；进入内核后是否调度其他任务取决于处理结果。

回答要点

1. 普通程序不能任意更改页表或直接控制设备，保护边界由硬件权限与内核共同维护；系统调用入口校验参数和权限，再把允许的操作转换成内核内部处理。
2. 外部设备完成、定时器等通常通过中断通知 CPU；除零、非法指令和缺页属于异常。不同体系结构对 trap、fault 等术语的细分不同，回答时应注明所用分类。
3. 系统调用有进入退出、参数检查和可能的数据传输成本，但库函数不必每次都触发系统调用；例如缓冲 I/O 可能先在用户空间完成，之后批量提交给内核。

常见追问

追问：为什么不能让应用直接操作硬件以提高速度？答：缺少受控隔离会让一个错误程序破坏其他任务或系统资源；高性能接口可减少调用次数或共享缓冲区，但仍需要权限、生命周期和资源控制协议。

易错点

易错点：认为进入内核态必然切换进程，或每个 C 库函数都是系统调用。权限级别变化、内核服务入口和调度切换应分别描述。

延伸阅读 / 原始参考资料

OSTEP • Limited Direct Execution

OSTEP • I/O Devices